Отлично. Я бы предложил не просто "Том 1", а оформить его как настоящий документ уровня корпоративной разработки. Ниже — начало такого документа. В нем пока нет деталей реализации таблиц (это будет Том 2), а описывается именно **идеология и фундамент системы**.

ТОМ 1

Концепция информационной системы технического надзора (TCMS)

Версия: 1.0
Статус: Концепция
Основа: Архитектурная философия Microsoft Access «Борей» (Northwind), адаптированная для веб-технологий и строительного технического надзора.

---

1. Назначение системы

1.1 Цель создания

Создать современную веб-информационную систему технического надзора (Technical Control Management System — TCMS), предназначенную для автоматизации процессов контроля строительства, ведения исполнительной документации, аналитики, подготовки отчетности и хранения истории объектов.

Система должна заменить разрозненные Excel-файлы, бумажные журналы и множество отдельных программ единым информационным пространством.

Основной принцип:

> **Одна система — один источник достоверных данных.**

---

2. Основная идея проекта

TCMS не является сайтом.

TCMS не является CRM.

TCMS не является обычной базой данных.

TCMS является корпоративной информационной системой (MIS), построенной по принципам Microsoft Access "Борей", но реализованной в виде современной веб-платформы.

---

3. Философия "Борея"

Microsoft Access Northwind ("Борей") стал популярным благодаря одной простой идее.

Пользователь никогда не работает с таблицами.

Он работает через формы.

Например

```
Карточка клиента
↓
Заказ
↓
Товар
↓
Отчет
```

Пользователь не знает SQL.

Не знает устройство базы.

Не знает структуру таблиц.

Он просто работает.

Именно эту философию необходимо полностью сохранить.

---

4. Философия TCMS

В TCMS пользователь никогда не работает с PostgreSQL.

Он работает только с:

- карточками;
- формами;
- мастерами;
- конструкторами;
- Dashboard.

Таким образом достигается максимальная простота использования.

---

5. Главный принцип

В системе существует только один источник истины.

```
PostgreSQL
```

Все остальные элементы получают информацию исключительно из базы данных.

Запрещается:

- хранить рабочие данные в Excel;
- использовать Excel как базу данных;
- использовать Google Sheets как основной источник данных;
- создавать дублирующие справочники.

Excel является исключительно форматом обмена информацией.

---

6. Архитектура системы

```
                    Пользователь

                          |

                   Web Browser

                          |

_____________________________________

                   Frontend

             React / Vue

_____________________________________

                    REST API
```

```
_____________________________________

                FastAPI

_____________________________________

            Business Logic Layer

_____________________________________

               PostgreSQL

_____________________________________

           File Storage

Фото

PDF

DWG

DOCX

XLSX
```

---

7. Архитектурная модель

Система разделяется на четыре независимых уровня.

Уровень представления

Отображает информацию пользователю.

Содержит:

- страницы;
- формы;
- карточки;
- Dashboard;
- таблицы;
- графики.

На этом уровне отсутствует бизнес-логика.

Уровень бизнес-логики

Самый важный уровень.

Выполняет:

- проверку данных;
- расчеты;
- сверку;
- импорт;
- экспорт;
- контроль доступа;
- обработку ошибок;
- формирование документов.

Любое изменение данных проходит только через бизнес-логику.

Уровень хранения данных

PostgreSQL хранит:

- справочники;
- рабочие данные;
- историю;
- документы;
- журнал изменений.

Уровень хранения файлов

Отдельное файловое хранилище.

Например

```
/Documents

/Photos

/Reports

/Acts

/Schemes

/Certificates
```

В PostgreSQL хранится только ссылка на файл.

---

8. Основные принципы системы

Принцип №1

Любая информация вводится только один раз.

Например.

ФИО сотрудника существует только в одном месте.

Во всех остальных разделах используется ссылка на запись.

Принцип №2

Любой документ строится автоматически.

Пользователь никогда не вводит повторно информацию, которая уже существует.

Например

```
АОСР

↓

Объект

↓

Подрядчик

↓

Ответственный инженер

↓

подставляются автоматически
```

Принцип №3

Любая запись имеет историю.

Например

```
Оклад

650000

↓

700000

↓

750000
```

История никогда не удаляется.

Принцип №4

Удаление данных запрещено.

Используется архивирование.

```
Активен

Архив

Удален логически
```

Это обеспечивает сохранность истории.

Принцип №5

Все действия пользователей записываются.

Например

```
Создал

Изменил

Удалил

Экспортировал

Импортировал
```

Утвердил

---

9. Основные объекты системы

Основными объектами являются не таблицы, а бизнес-сущности.

Например.

```
Сотрудник
Проект
Объект
Пятно
Блок
Этаж
Подрядчик
Замечание
Инспекция
Документ
Испытание
АОСР
Материал
Исполнительная схема
Фото
Пользователь
Организация
```

Каждая сущность имеет собственную карточку.

---

10. Карточка объекта

Карточка является центральным элементом системы.

Например.

```
ЖК JAMBYL
↓
Информация
↓
Пятна
↓
Блоки
↓
Этажи
↓
Подрядчики
↓
Замечания
↓
Документы
↓
Фото
↓
АОСР
↓
Испытания
↓
История
```

Карточка отображает не одну таблицу, а всю связанную информацию.

11. Навигация

Вместо открытия таблиц пользователь работает через разделы.

```
Главная

Справочники

Объекты

Проекты

Инспекции

Замечания

Документы

Исполнительная документация

АОСР

Испытания

Переработки

KPI

Dashboard

Отчеты

Администрирование
```

12. Импорт данных

Любой импорт проходит одинаковые этапы.

```
Выбор файла

↓

Определение шаблона

↓

Чтение Excel
```

```
↓

Нормализация

↓

Проверка

↓

Сверка

↓

Показ различий

↓

Подтверждение

↓

Запись в PostgreSQL
```

Пользователь никогда не работает напрямую с базой.

---

13. Конструктор документов

Любой документ строится по одному принципу.

```
Выбор шаблона

↓

Выбор проекта

↓

Выбор периода

↓

Автоматическая подстановка данных

↓

Ввод недостающей информации
```

```
↓

Автоматические расчеты

↓

Получение Excel

или PDF
```

---

14. Конструктор отчетов

Отчеты создаются без знания SQL.

Пользователь выбирает:

- источник данных;
- поля;
- фильтры;
- группировку;
- сортировку;
- итоговые вычисления.

Система автоматически формирует SQL-запрос, выполняет его и отображает результат в виде таблицы, графика или файла Excel/PDF.

---

15. Бесплатный технологический стек

Одним из ключевых требований проекта является использование только бесплатного программного обеспечения с открытой лицензией, чтобы исключить затраты на лицензии и зависимость от одного поставщика.

Компонент	Технология	Причина выбора
База данных	**PostgreSQL**	Бесплатная, надежная, промышленный уровень, поддержка сложных запросов и расширений.
Backend	**Python + FastAPI**	Высокая производительность, автоматическая документация API (OpenAPI), простота разработки.
ORM	**SQLAlchemy**	Работа с PostgreSQL через объектную модель, поддержка миграций.

Компонент	Технология	Причина выбора
Миграции БД	**Alembic**	Контроль версий структуры базы данных.
Frontend	**Vue.js** (или React)	Современный интерфейс, компонентная архитектура. Для первого этапа я рекомендую Vue.js как более простой для освоения.
UI-компоненты	**Vuetify** (для Vue) или **PrimeVue**	Готовые таблицы, формы, диалоги, дерево объектов.
Авторизация	**JWT + OAuth2**	Стандартная безопасная аутентификация.
Сервер	**Nginx**	Бесплатный веб-сервер и обратный прокси.
Контейнеризация	**Docker + Docker Compose**	Простое развертывание на любом сервере.
Очереди задач (при необходимости)	**Celery + Redis**	Генерация тяжелых отчетов, массовый импорт, фоновые задачи.
Хранение файлов	Локальная файловая система (на первом этапе)	Простое и бесплатное решение. В дальнейшем можно заменить на MinIO или другое S3-совместимое хранилище.
Экспорт Excel	**openpyxl**	Бесплатная библиотека для создания и редактирования XLSX.
Экспорт Word	**python-docx**	Генерация документов DOCX по шаблонам.
Экспорт PDF	**WeasyPrint** или **ReportLab**	Формирование печатных документов без коммерческих библиотек.
Диаграммы	**Chart.js**	Бесплатные интерактивные графики для Dashboard.
Карты (при необходимости)	**Leaflet + OpenStreetMap**	Полностью бесплатная альтернатива коммерческим картографическим сервисам.
Контроль версий	**Git**	Бесплатное управление исходным кодом.
Репозиторий	**GitHub** или **GitLab Community Edition**	Совместная разработка и хранение кода.

16. Принципы развития системы

Архитектура должна предусматривать развитие без переписывания ядра.

Планируемые направления:

- мобильное приложение для инженеров технического надзора;
- интеграция с BIM-моделями;
- интеграция с электронным документооборотом;
- интеграция с лабораторными испытаниями;
- интеграция с системами учета (например, 1С или ERP);
- API для обмена данными с внешними системами;
- модуль уведомлений (e-mail, Telegram и др.).

Заключение

TCMS не является веб-сайтом в классическом понимании. Это веб-версия концепции Microsoft Access «Борей», адаптированная для строительной отрасли и технического надзора.

Главные принципы проекта:

- пользователь работает с **карточками, формами и конструкторами**, а не с таблицами;
- **PostgreSQL** — единственный источник достоверных данных;
- **Excel, Word и PDF** — только форматы обмена и представления информации;
- **вся бизнес-логика сосредоточена на сервере**, что обеспечивает единообразие работы и контроль качества данных;
- используются **только бесплатные технологии с открытым исходным кодом**, исключающие лицензионные расходы и обеспечивающие возможность долгосрочного развития системы.

Предложение перед Томом 2

Прежде чем переходить к структуре PostgreSQL, я рекомендую добавить еще один документ — **«Том 1A. Методология проектирования TCMS»**. В нем будут описаны правила именования таблиц, полей, идентификаторов (ID), принципы аудита, версионирования, архивирования и проектирования карточек. Это станет единым стандартом, по которому будут создаваться все модули системы и база данных.

Именно такой документ обычно разрабатывается перед проектированием промышленной СУБД и значительно упрощает дальнейшую разработку.